decoding and language models on tasks that need longer-horizon reasoning and planning.

7 Future Work

In current work, the abstract latent prediction horizon is a manually set hyperparameter, which limits the flexibility of lookahead planning. Further designs including dynamically choosing the lookahead horizon could be beneficial. Also, the NTP head is used during inference in the current implementation, but combined NTP can also be used if we trade speed for accuracy.

References

Kwangjun Ahn, Alex Lamb, and John Langford. 2025. Efficient joint prediction of multiple future tokens. *arXiv preprint arXiv:2503.21801*.

Gregor Bachmann and Vaishnavh Nagarajan. 2024. The pitfalls of next-token prediction. *arXiv preprint arXiv:2403.06963*.

Pierre-Luc Bacon, Jean Harb, and Doina Precup. 2017. The option-critic architecture. In *Proceedings of the AAAI conference on artificial intelligence*, volume 31.

Loïc Barrault, Paul-Ambroise Duquenne, Maha Elbayad, Artyom Kozhevnikov, Belen Alastruey, Pierre Andrews, Mariano Coria, Guillaume Couairon, Marta R Costa-jussà, David Dale, and 1 others. 2024. Large concept models: Language modeling in a sentence representation space. *arXiv preprint arXiv:2412.08821*.

Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, and 39 others. 2021. Evaluating large language models trained on code.

Shizhe Diao, Yu Yang, Yonggan Fu, Xin Dong, Dan Su, Markus Kliegl, Zijia Chen, Peter Belcak, Yoshi Suhara, Hongxu Yin, and 1 others. 2026. Nemotron-climb: Clustering-based iterative data mixture bootstrapping for language model pre-training. *Advances in Neural Information Processing Systems*, 38.

Leo Gao, Jonathan Tow, Baber Abbasi, Stella Biderman, Sid Black, Anthony DiPofi, Charles Foster, Laurence Golding, Jeffrey Hsu, Alain Le Noac'h, Haonan Li, Kyle McDonell, Niklas Muennighoff, Chris Ociepa, Jason Phang, Laria Reynolds, Hailey Schoelkopf, Aviya Skowron, Lintang Sutawika, and 5 others. 2024. The language model evaluation harness.

Fabian Gloeckle, Badr Youbi Idrissi, Baptiste Rozière, David Lopez-Paz, and Gabriel Synnaeve. Better & faster large language models via multi-token prediction, 2024. *URL https://arxiv. org/abs/2404.19737*.

Sukjun Hwang, Brandon Wang, and Albert Gu. 2025. Dynamic chunking for end-to-end hierarchical sequence modeling. *arXiv preprint arXiv:2507.07955*.

Lingjing Kong, Guangyi Chen, Biwei Huang, Eric Xing, Yuejie Chi, and Kun Zhang. 2024. Learning discrete concepts in latent hierarchical models. *Advances in Neural Information Processing Systems*, 37:36938–36975.

Jeffrey Li, Alex Fang, Georgios Smyrnis, Maor Ivgi, Matt Jordan, Samir Gadre, Hritik Bansal, Etash Guha, Sedrick Keh, Kushal Arora, Saurabh Garg, Rui Xin, Niklas Muennighoff, Reinhard Heckel, Jean Mercat, Mayee Chen, Suchin Gururangan, Mitchell Wortsman, Alon Albalak, and 40 others. 2025. Datacomp-lm: In search of the next generation of training sets for language models. *Preprint*, arXiv:2406.11794.

Divyat Mahajan, Sachin Goyal, Badr Youbi Idrissi, Mohammad Pezeshki, Ioannis Mitliagkas, David Lopez-Paz, and Kartik Ahuja. 2025. Beyond multi-token prediction: Pretraining llms with future summaries. *arXiv preprint arXiv:2510.14751*.

Vaishnavh Nagarajan, Chen Henry Wu, Charles Ding, and Aditi Raghunathan. 2025. Roll the dice & look before you leap: Going beyond the creative limits of next-token prediction. *Preprint*, arXiv:2504.15266.

Artidoro Pagnoni, Ramakanth Pasunuru, Pedro Rodriguez, John Nguyen, Benjamin Muller, Margaret Li, Chunting Zhou, Lili Yu, Jason E Weston, Luke Zettlemoyer, and 1 others. 2025. Byte latent transformer: Patches scale better than tokens. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 9238–9258.

Chenze Shao, Darren Li, Fandong Meng, and Jie Zhou. 2025. Continuous autoregressive language models. *arXiv preprint arXiv:2510.27688*.

Richard S Sutton, Doina Precup, and Satinder Singh. 1999. Between mdps and semi-mdps: A framework for temporal abstraction in reinforcement learning. *Artificial intelligence*, 112(1-2):181–211.

Xiaoyu Tao, Tingyue Pan, Mingyue Cheng, Yucong Luo, Qi Liu, and Enhong Chen. 2024. Hierarchical multimodal llms with semantic space alignment for enhanced time series classification. *ACM Transactions on Intelligent Systems and Technology*.

Jayden Teoh, Manan Tomar, Kwangjun Ahn, Edward S Hu, Tim Pearce, Pratyusha Sharma, Akshay Krishnamurthy, Riashat Islam, Alex Lamb, and John Langford. 2025. Next-latent prediction transformers learn compact world models. *arXiv preprint arXiv:2511.05963*.

Lili Yu, Dániel Simig, Colin Flaherty, Armen Aghajanyan, Luke Zettlemoyer, and Mike Lewis. 2023. Megabyte: Predicting million-byte sequences with multiscale transformers. *Advances in Neural Information Processing Systems*, 36:78808–78823.

Wancong Zhang, Basile Terver, Artem Zholus, Soham Chitnis, Harsh Sutaria, Mido Assran, Randall Balestriero, Amir Bar, Adrien Bardes, Yann LeCun, and 1 others. 2026. Hierarchical planning with latent world models. *arXiv preprint arXiv:2604.03208*.

A HiLP Training Procedure

Algorithm 1 summarizes one HiLP training step, combining the five objectives into a single backward pass. The SWA module is a causal sliding-window self-attention over the lower latents, so every position t carries its own higher-level latent $\mathbf{u}_t$ summarizing the last k lower latents. All latent-space prediction targets are stop-gradiented, and both distributions in the KL term are decoded through a frozen copy of the NTP head (p_θ^{sg}), so each auxiliary loss shapes the representation only through its designated pathway: the rollout losses reach $\mathbf{h}$ through p_ψ, the higher-level consistency loss trains p_ϕ and the SWA module, and the combined cross-entropy trains p_ρ and the SWA module alone via the stop-gradient inside $\tilde{\mathbf{u}}_t$. At inference, only G_θ and p_θ are kept: the SWA module, both dynamics models, and the combined head are discarded.

B Experiment hyperparameters

Algorithm 1 One HiLP training step. $E(\cdot)$ denotes the backbone, k the SWA window size and lookahead offset, and d the rollout depth. Expectations over t average over all valid positions in the batch.

Require: batch $X_{1:T}$; backbone G_θ with NTP head p_θ; lower dynamics p_ψ; SWA module; higher dynamics p_ϕ; combined head p_ρ; loss weights $\lambda_{\text{ntp}}, \lambda_{\text{h}}, \lambda_{\text{KL}}, \lambda_{\text{u}}, \lambda_{\text{cntp}}$

$\mathbf{h}_{1:T} \leftarrow G_\theta(X_{1:T})$ ▷ one backbone pass

$\mathcal{L}_{\text{ntp}} \leftarrow \mathbb{E}_t\big[-\log p_\theta(X_{t+1} \mid \mathbf{h}_t)\big]$ ▷ Eq. (A)

Lower-level rollout (teacher-forced tokens):

$\hat{\mathbf{h}}_t \leftarrow \mathbf{h}_t$ for all t; $\quad \mathcal{L}_{\text{h}}, \mathcal{L}_{\text{KL}} \leftarrow 0$

for $i = 1, \ldots, d$ **do**

  $\hat{\mathbf{h}}_{t+i} \leftarrow p_\psi\big(\hat{\mathbf{h}}_{t+i-1},\ E(X_{t+i})\big)$

  $\mathcal{L}_{\text{h}} \mathrel{+}= \frac{1}{d}\,\mathbb{E}_t\big[\text{SL1}\big(\text{sg}[\mathbf{h}_{t+i}],\ \hat{\mathbf{h}}_{t+i}\big)\big]$ ▷ Eq. (B)

  $\mathcal{L}_{\text{KL}} \mathrel{+}= \frac{1}{d}\,\mathbb{E}_t\big[D_{\text{KL}}\big(p_\theta^{\text{sg}}(\cdot \mid \text{sg}[\mathbf{h}_{t+i}]) \,\big\|\, p_\theta^{\text{sg}}(\cdot \mid \hat{\mathbf{h}}_{t+i})\big)\big]$ ▷ Eq. (C)

end for

Higher-level channel:

$\mathbf{u}_t \leftarrow \text{SWA}(\mathbf{h}_{t-k+1:t})$ ▷ abstract latent

$\hat{\mathbf{u}}_{t+k} \leftarrow p_\phi(\mathbf{u}_t)$ ▷ k-step lookahead, no token input

$\mathcal{L}_{\text{u}} \leftarrow \mathbb{E}_t\big[\text{SmoothL1}\big(\text{sg}[\mathbf{u}_{t+k}],\ \hat{\mathbf{u}}_{t+k}\big)\big]$ ▷ Eq. (D)

Combined head (sg keeps $\mathbf{h}$ *governed by NTP and rollout):*

$\tilde{\mathbf{u}}_t \leftarrow \text{SWA}\big(\text{sg}[\mathbf{h}_{t-k+1:t}]\big)$

$\mathcal{L}_{\text{cntp}} \leftarrow \mathbb{E}_t\big[-\log p_\rho(X_{t+1} \mid \mathbf{h}_t, \tilde{\mathbf{u}}_t)\big]$ ▷ Eq. (E)

$\mathcal{L} \leftarrow \lambda_{\text{ntp}}\mathcal{L}_{\text{ntp}} + \lambda_{\text{h}}\mathcal{L}_{\text{h}} + \lambda_{\text{KL}}\mathcal{L}_{\text{KL}} + \lambda_{\text{u}}\mathcal{L}_{\text{u}} + \lambda_{\text{cntp}}\mathcal{L}_{\text{cntp}}$

One optimizer step on $\nabla\mathcal{L}$ w.r.t. θ, ψ, ϕ, ρ, and the SWA module

C Latent overhead and efficiency.

Table 5 compares the four 100B-token code-data runs along three axes: total parameter count, the subset of parameters actually used during greedy NTP decoding, and training throughput. All four models share the same 1.06 B-parameter trunk and tied-free LM head, so the verifier path used at inference is identical and the additional parameters in MTP, NextLat, and HiLP are auxiliary draft components that are not required to produce the next-token distribution. NextLat adds a small (≈19 M) next-latent predictor on top of the trunk, while HiLP and MTP each add a ≈190–200 M draft module. Training throughput scales inversely with the size of the auxiliary loss graph: the NTP baseline reaches ≈126 K tokens/s/GPU, NextLat retains ≈83% of that throughput, HiLP retains ≈65%, and MTP drops to ≈49% because its four future-token heads must each be evaluated against the LM logits at every step.

D Input-Combination Modes

We include an implementation-level ablation comparing the historical `concat` input-combination mode against the current `glu_cross` mode used by latent prediction modules. Let D denote the hidden size, m the latent-head hidden multiplier, and V the vocabulary size. In the lower next-latent predictor, `concat` forms $[\mathbf{h}_t; e_{t+1}] \in \mathbb{R}^{2D}$ and feeds it directly to the SwiGLU predictor. This gives an intermediate width of $2mD$ and costs $10mD^2$ parameters and multiply-adds per token for the gate, up, and down projections, ignoring biases. By contrast, `glu_cross` computes

$$W_h\mathbf{h}_t \odot \sigma(W_e e_{t+1}) \in \mathbb{R}^D,$$

adding two $D \times D$ projections but reducing the following SwiGLU width to mD. Its corresponding cost is therefore $(3m+2)D^2$. With the default $m = 4$, this is $14D^2$ versus $40D^2$, or about 35% of the `concat` predictor cost.

The same distinction appears in the combined NTP head. For `concat`, the combined head maps $[\mathbf{h}_t; \mathbf{u}_t] \in \mathbb{R}^{2D}$ to logits, costing $2DV$. For `glu_cross`, the gated fusion costs $2D^2$ and the logits are produced from a D-dimensional vector, costing DV, for a total of $DV+2D^2$. Thus `glu_cross` is especially attractive when $V \gg D$: it approximately halves the combined-head logit computation while also reducing the latent predictor from a $2D$-wide to a D-wide SwiGLU input.

Group	Hyperparameter	Value
Backbone model	Vocabulary size	100,352
	Training sequence length	8,192
	Hidden layers	24
	Attention heads	16
Architecture	Latent input combination	GLU-cross
	Hidden multiplier	2
	Latent window size	4
	Latent lookahead steps	4
Loss weights	NTP loss weight	1.0
	MSE loss weight	10.0
	KL loss weight	0.1
	Higher-level MSE loss weight	1.0
	Combined NTP loss weight	0.5
Combined head	Input combination	GLU-cross
	Downstream LM head	NTP
	Draft LM head	NTP
Optimization	Latent learning rate	1.0×10^{-3}
Latent evaluation	Evaluation horizon	8

Table 4: HiLP hyperparameter configuration.

Model	# Params	Inf. Params	Training Step Breakdown (ms)				Training Throughput			
			Step	Fwd	Bwd	Optim	Tok/s/GPU↑	TF/GPU↑	MFU↑	Samp/s↑
NTP	1.06 B	1.06 B	449	119	293	37	126,278	898	0.399	123.3
MTP	1.25 B	1.06 B	916	631	243	43	61,789	509	0.226	60.3
NextLat	1.08 B	1.06 B	542	205	295	42	104,505	884	0.393	102.1
HiLP	1.27 B	1.06 B	690	329	315	46	81,653	780	0.347	79.7

Table 5: Parameter counts and training efficiency for the 100B-token runs on 8×B200 GPUs. **# Params**: total trained parameters. **Inf. Params**: verifier path (trunk + LM head) used during greedy NTP decoding, identical across all models. **Step breakdown**: `timing/step_time_sec` and its components (forward, backward, optimizer), in milliseconds. **Throughput**: tokens/s/GPU, TFLOP/s/GPU, model FLOPs utilization, and samples/s (all GPUs).

Mode	NTP loss↓	Step (ms)↓
`concat`	4.113	919.5
`glu_cross`	3.942	818.9

Table 6: Input-combination ablation comparing next-token prediction loss and training step time.

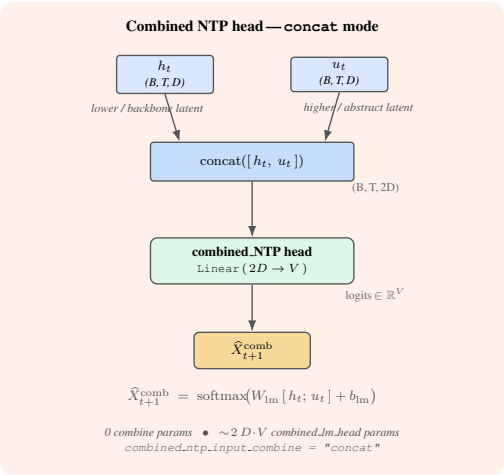


Figure 4: `concat` input-combination mode used by the latent and combined NTP heads.

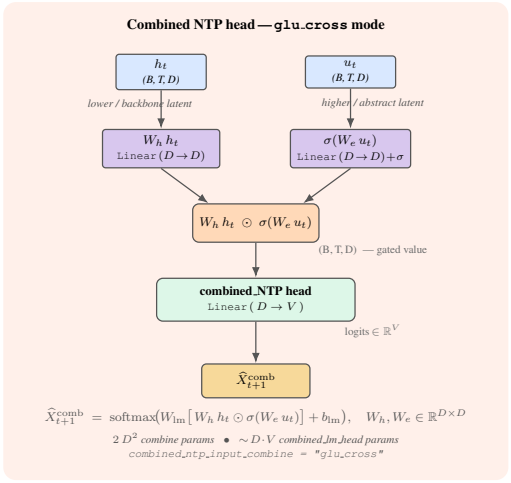


Figure 5: `glu_cross` input-combination mode, which gates a projected lower latent with the conditioning latent while preserving hidden width D.